System Architecture & Implementation Guide

JIRA MEAL-5 • Version 1.3 • 01 November 2026

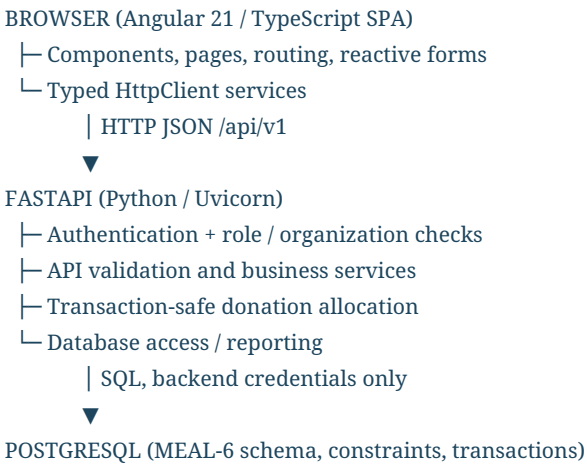
Working technical reference for frontend, backend and database implementation. This guide describes the agreed MVP boundaries and the practical conventions to follow during development.

1. Purpose and system overview

Meal Connect coordinates safe, timely surplus-food donations from restaurants and bakeries to verified NGOs, with volunteers/collectors supporting pickups and NGOs recording distribution. It is a locally developed web MVP. Beneficiaries do not have accounts; NGOs coordinate distribution on their behalf.

Area	Implementation baseline
Frontend	Angular 21 SPA, TypeScript, Angular Routing and SCSS
Backend	Python FastAPI REST API
Database	PostgreSQL, accessed only by backend
Local run	Angular dev server + Uvicorn + PostgreSQL, without Docker
Source control	Git feature branches → Pull Request → develop; main remains stable
Out of scope	Mobile apps, payments, AI matching, live GPS, external delivery platforms, production hosting

2. Architecture and boundaries



Layer	Responsibilities	Never do
Angular UI	Render pages; forms; call typed API services; accessible error feedback	Connect to PostgreSQL; trust UI permission checks as security

FastAPI	Authenticate; authorize; validate; implement workflow and API errors	Trust client role/stock/status as authoritative
PostgreSQL	Enforce PK/FK/UNIQUE/CHECK, store workflow facts, protect allocations	Expose credentials to frontend

3. Users and responsibilities

Actor	Allowed business functions
Restaurant / Bakery	Create and maintain own food listings; approve whole NGO request or reject
Verified NGO	Browse eligible donations; request quantity from an existing listing; record received distribution
Volunteer / Collector	See assigned pickup; update authorized pickup/delivery states
Administrator	Verify organizations, manage user/organization issues and view authorized reports

An organization's verification_status and user membership/role are checked in FastAPI for every protected operation. Angular route guards improve navigation only; they are not authorization.

4. Donation lifecycle (MVP rules)

- Business publishes a donation with total quantity, food details and pickup deadline; listing becomes available when eligible.
- Verified NGO opens that existing listing and submits requested_quantity. The NGO cannot publish a standalone general food request.
- Business owner decides the request: approve its entire requested quantity or reject it. Partial approval is not supported.
- On approval FastAPI atomically verifies available quantity and request/listing state, updates allocated_quantity and request state, and records history.
- Assigned collector updates pickup/delivery; NGO records completed distribution. Reports use confirmed records, not hypothetical demand.

Mandatory consistency rules

- requested_quantity must be positive; approvals must not make allocated_quantity exceed total_quantity.
- An expired or cancelled donation cannot accept new requests or approvals.
- Concurrent approvals must be serialized or guarded by transactional conditional updates/row locking; exactly one safe outcome commits.
- Only the owning business can approve/reject; verified NGO can request; assigned collector can update their pickup.
- Every workflow state change is validated against actual database enums and captured in status_history where applicable.
- MEAL-6 reviewed DBML is the logical data model; implement real PostgreSQL constraints and indexes through migrations.

5. Data model integration

Use the 10 MEAL-6 entities: users, organizations, organization_memberships, organization_verifications, donations, donation_requests, pickups, distribution_records, notifications and status_history. Keep identifiers and state names aligned to the reviewed DBML, not to older sample diagrams.

Database rule	Implementation instruction
Relationships	PK/FK connect organizations, listings, requests, pickups and distribution
Single pickup per request	UNIQUE constraint on pickups.request_id
Single distribution per pickup	UNIQUE constraint on distribution_records.pickup_id
Quantity validation	CHECK positive values and bounded totals where schema supports; cross-row allocation remains transactional
State values	Use MEAL-6 enums (e.g., available, partially_allocated, fully_allocated); never invent frontend-only statuses

6. API contract to implement

Use JSON UTF-8, snake_case fields and REST routes under /api/v1. The catalogue below is the implementation starting contract. Match exact request/response models to the MEAL-6 DBML and MEAL-62 before coding each endpoint. No endpoint is represented as already built.

Method / path (relative to /api/v1)	Actor / behavior
POST /auth/register; POST /auth/login	Register / authenticate
GET /organizations/me	Read own organization data / verification
POST /organizations/verification-requests	Submit verification details
GET /donations	Browse eligible donations with filters and pagination
POST /donations; PATCH /donations/{donation_id}	Business creates / edits own listing
POST /donations/{donation_id}/requests	NGO requests quantity from listing
POST /requests/{request_id}/approve	Owning business approves whole request transactionally
POST /requests/{request_id}/reject	Owning business rejects request
PATCH /pickups/{pickup_id}	Assigned collector updates pickup state
POST /pickups/{pickup_id}/distribution	Authorized NGO records distribution
GET /reports/impact	Authorized aggregate report

Responses and error conventions

HTTP: 200 read/action, 201 create, 204 empty success, 400 malformed, 401 unauthenticated, 403 forbidden, 404 absent/inaccessible, 409 conflict, 422 validation, 500 unexpected. FastAPI exception handlers should normalize framework validation output to the MEAL-62 error format.

201 Created

{"data":{"id":101,"status":"available"},"message":"Donation created"}

409 Conflict

{"error":{"code":"QUANTITY_CONFLICT","message":"The requested quantity is no longer available","details":[],"request_id":"abc123"}}

422 Validation

{"error":{"code":"VALIDATION_ERROR","message":"Please correct the highlighted fields","details":[{"field":"requested_quantity","message":"Must be greater than zero"}],"request_id":"abc123"}}

Required error properties: error.code and error.message. Optional details and request_id should be populated consistently by a shared handler. Never return stack traces or credentials in client responses.

7. Angular 21 implementation guidelines

- Build page components by feature (auth, donations, requests, pickups, distribution, admin) and keep shared UI in a shared area.
- Keep HTTP calls in injectable Angular services using HttpClient, not duplicated in components; define TypeScript interfaces for the actual JSON contracts.
- Use Angular Router and guards for navigation; validate roles/ownership again on every FastAPI protected endpoint.
- Use reactive forms for donation and quantity inputs; show 422 field errors and 409 quantity conflicts clearly; refresh affected listing after conflicts.
- Set API base URL in frontend configuration; never put database credentials or backend signing secrets in the SPA.
- Do not manually recreate legacy frontend/assets, frontend/css or frontend/js directories. Use the Angular CLI-generated layout and frontend/public for static assets as appropriate.

8. FastAPI and PostgreSQL implementation guidelines

- Separate routers, Pydantic schemas, business services and database access; do not implement transaction logic directly inside Angular.
- Use a Python virtual environment, explicit dependency versions and a local PostgreSQL database; preserve secrets in a local .env ignored by Git.
- Create database migrations for MEAL-6 entities, FK/UNIQUE/positive checks and indexes. Confirm exact column/enum names against DBML.
- Implement role + verification + ownership rules centrally; write status_history entries when relevant workflow states change.
- Protect approval in one database transaction. If insufficient quantity or invalid state, return 409 without modifying allocated_quantity.
- Document endpoints through FastAPI /docs during local development; include basic automated tests for authorization, concurrency and lifecycle.

9. Run locally and connect components

Service	Local development guidance

PostgreSQL	Start locally; configure DB URL only in backend environment
FastAPI	Activate backend venv; run Uvicorn on 127.0.0.1:8000 when backend entry point is implemented
Angular	cd frontend, npm install (or npm ci from lockfile), npm start; typically localhost:4200
CORS	Allow exact Angular development origin(s) in FastAPI, not a wildcard with credentials
Verification	Open Angular page; inspect API calls; verify FastAPI /docs; execute request/approval flow against local database

The Angular 21 default welcome page has already loaded on localhost; that confirms frontend bootstrapping only, not backend connectivity or full application functionality. Exact Node/npm/Angular CLI/Python/PostgreSQL version compatibility is recorded in MEAL-163.

10. Suggested development sequence

- Establish backend entry point, local database and migrations from MEAL-6.
- Create user/organization authentication and verification workflow.
- Implement donation listing + NGO quantity request + all-or-nothing approval transaction.
- Connect Angular services/forms to API routes and handle standardized errors.
- Implement pickup, distribution and notifications, then aggregate reports.
- Test expiry, cancellation, ownership, verification and simultaneous approval conflicts.

11. Related technical documents

MEAL-61: high-level component diagram • MEAL-62: canonical API/error conventions • MEAL-63: backend environment • MEAL-163: prerequisites/versions • MEAL-6: data model and allocation constraints • MEAL-197: project implementation overview • MEAL-209: risk and decision log • MVP Scope v1.1: functional boundaries.

Document maintenance: if a route, schema or architecture rule changes during development, update the relevant technical guide and implementation in the same change. Do not describe a design example as a deployed feature.